

Trabajo Práctico — La aplicación en internet

Consigna de trabajo práctico · Unidad de Despliegue en un VPS

Trabajo Práctico — La aplicación en internet	1
1. Encuadre: qué hay que lograr	1
2. Condiciones de trabajo	2
2.1. Grupo e infraestructura	2
2.2. Reparto del trabajo	3
3. El sistema que hay que desplegar	3
4. Requisitos obligatorios	4
R1 — Dominio propio con resolución correcta	4
R2 — VPS asegurada	4
R3 — Imágenes construidas desde el repositorio	4
R4 — Publicación con HTTPS válido	4
R5 — El contrato funciona de punta a punta	5
R6 — Persistencia en base de datos aislada	5
R7 — Degradación elegante	5
5. Protocolo de verificación	5
5.1. Resolución de nombres	5
5.2. Superficie de exposición	5
5.3. Transporte	6
5.4. Contrato y errores	6
5.5. Persistencia y aislamiento	6
5.6. Degradación	6
6. El informe	6
7. Errores frecuentes	7
8. Entrega	7
9. Preguntas frecuentes	8
10. Anexo — Lista de control previa a la entrega	8

1. Encuadre: qué hay que lograr

Las cinco clases de la unidad dejaron, cada una, una pieza funcionando: un nombre que resuelve, un servidor accesible y cerrado, dos imágenes que se construyen solas, un despliegue con certificado válido y una base de datos que nadie alcanza desde afuera. Este trabajo práctico **no agrega tecnología nueva**: pide que esas cinco piezas estén simultáneamente en pie, sobre la misma infraestructura, y que el grupo pueda explicar por qué cada una está donde está.

La distinción es importante y conviene enunciarla desde el principio. Un despliegue que funciona el día de la demostración no prueba demasiado: puede funcionar por casualidad, con un puerto

abierto de más, una credencial versionada o un certificado que vence en tres días. Lo que se busca acá es **un sistema que funciona por las razones correctas**, y esa diferencia se detecta con las mismas herramientas de diagnóstico que la unidad enseñó a usar. Por eso el protocolo de verificación de la sección 5 está publicado de antemano: no es una sorpresa, es la definición operativa de “terminado”.

Contenidos de esta consigna

1. Encuadre y objetivo del trabajo.
2. Condiciones de trabajo: grupo, infraestructura y plazos.
3. El sistema que hay que desplegar.
4. Requisitos obligatorios, uno por uno.
5. Protocolo de verificación.
6. El informe.
7. Errores frecuentes.
8. Entrega: qué, cómo y cuándo.
9. Preguntas frecuentes.
10. Anexo: lista de control previa a la entrega.

PARA ENTENDER

Leé la sección 5 **antes** de empezar, no al final. Ahí está, comando por comando, cómo se comprueba que el despliegue está terminado. Un trabajo donde conocés las comprobaciones desde el día uno no es un trabajo más fácil: es uno donde no te queda nada sin verificar.

2. Condiciones de trabajo

2.1. Grupo e infraestructura

Elemento	Condición
Integrantes	De 3 a 6 personas. No se aceptan entregas individuales ni de dos.
VPS	Una por grupo, o una compartida entre hasta 3 grupos , con Easypanel y mínimo 2 GB de RAM. Cada grupo trabaja en su propio proyecto.
Dominio	Uno solo por grupo , obtenido con el GitHub Student Developer Pack
Repositorios	Los dos que provee la cátedra: uno del backend y uno del frontend
Acceso del docente	El docente y todos los integrantes deben tener acceso al proyecto en el panel.

El dominio corre por cuenta de un integrante, pero todos tienen que haber completado **el trámite del Student Pack**. El motivo es doble: el beneficio es personal y les queda para sus propios proyectos, y si el dominio elegido tiene un problema —registros CAA heredados, una zona DNS que el proveedor demora en propagar— hay otras alternativas disponibles sin perder una semana.

Si varios grupos comparten una VPS, cada uno crea su propio proyecto en Easypanel: sus tres servicios, sus propias variables de entorno y su propio dominio apuntando a la misma IP. Lo que se comparte es la máquina, y con ella el firewall, el acceso SSH y el estado de los puertos: las condiciones del R2 pasan a ser **responsabilidad conjunta**, porque si un grupo publica el 5432, el barrido lo ve abierto para todos.

2.2. Reparto del trabajo

El trabajo es grupal, pero **el aprendizaje es individual**: la unidad se cierra bien cuando cada integrante puede explicar cualquier parte del sistema. Un grupo donde una sola persona hizo todo el despliegue y el resto miró termina con una sola persona que aprendió a desplegar.

La recomendación operativa es rotar: que cada integrante ejecute al menos una de las cinco piezas de punta a punta y explique al resto qué hizo. Es más lento y es el único reparto que sobrevive a la pregunta “¿por qué el puerto 5432 no está publicado?” hecha al azar.

⚠ OJO ACÁ

No compartan la clave privada de SSH entre todos. Una clave privada es personal, como una contraseña. Lo correcto es que cada integrante genere su propio par y que las claves públicas de todos se carguen en el servidor, que para eso admite varias. Si comparten una sola, el día que alguien deja el grupo hay que rotar todo, y además ninguna acción del servidor queda atribuible a nadie.

3. El sistema que hay que desplegar

El resultado es la aplicación de tres servicios que provee la cátedra, desplegada sobre una VPS, publicada bajo un dominio propio con certificado válido, con la base de datos inalcanzable desde internet.

Servicio	Qué es	Alcance desde internet	Nombre interno
web	Nginx sirviendo el <code>index.html</code> del frontend	Sí, por <code>tudominio.com</code>	—
api	FastAPI, la lógica de cálculo y el historial	Sí, por <code>api.tudominio.com</code>	<code>calculadora_api</code>
db	PostgreSQL con el historial de operaciones	No, ningún puerto publicado	<code>calculadora_db</code>

La configuración de los tres servicios se hace **por variables de entorno**, sin editar código para desplegar:

Servicio	Variable	Valor esperado
web	<code>API_URL</code>	<code>https://api.tudominio.com</code>
api	<code>ORIGENES_PERMITIDOS</code>	<code>https://tudominio.com</code> (el origen exacto del frontend)
api	<code>DATABASE_URL</code>	La cadena de conexión al servicio db, con nombre interno

 PARA ENTENDER

Fijate que ninguna de las tres variables contiene una dirección IP. `API_URL` lleva un nombre de dominio y `DATABASE_URL` lleva un nombre de servicio de la red interna. Es la misma idea de la sección 5.2 del material aplicada dos veces: **un nombre estable delante de una dirección que puede cambiar**. Si en alguna de las tres escribiste una IP, funciona hoy y se rompe el día que algo se redesplice en otro orden.

4. Requisitos obligatorios

Son siete. Cada uno indica qué se pide, cómo se verifica y en qué sección del material está fundamentado. **Todos son obligatorios: el trabajo se da por terminado cuando los siete pasan.**

R1 — Dominio propio con resolución correcta

El dominio del grupo debe resolver a la IP de la VPS, y el subdominio `api` también. Se acepta un registro comodín (*) o dos registros A explícitos; lo que no se acepta es el dominio automático que genera el panel, porque el módulo entero existe para no depender de él.

Verificación: `dig +short tudominio.com` y `dig +short api.tudominio.com` devuelven la IP de la VPS. **Fundamento:** Clase 1, secciones 1.4 a 1.7.

R2 — VPS asegurada

Condición	Estado exigido
Acceso SSH por clave pública	Habilitado, con las claves de los integrantes
Acceso SSH por contraseña	Deshabilitado
Firewall	Activo, con política restrictiva por defecto
Puertos alcanzables desde internet	Únicamente 22, 80 y 443

Verificación: un barrido de puertos desde afuera no debe mostrar nada más que esos tres. **Fundamento:** Clase 2, secciones 2.5 a 2.9.

R3 — Imágenes construidas desde el repositorio

Los dos servicios de aplicación se construyen a partir del `Dockerfile` que vive en la raíz de cada repositorio provisto por la cátedra. **No se acepta** haber subido archivos al servidor por SFTP, ni haber editado nada dentro del contenedor: si la imagen no sale del repositorio, el despliegue no es reproducible y el requisito no está cumplido.

Verificación: el panel muestra el origen del build y el commit desplegado; el registro de construcción muestra la ejecución del `Dockerfile`. **Fundamento:** Clase 3, secciones 3.4 a 3.8.

R4 — Publicación con HTTPS válido

Los dos servicios públicos responden por HTTPS con certificado emitido por una autoridad reconocida, y el acceso por HTTP redirige a HTTPS. Un certificado autofirmado, o una advertencia del navegador de cualquier tipo, incumple el requisito.

Verificación: `curl -I https://tudominio.com` devuelve 200 sin `--insecure`, y `curl -I http://tudominio.com` devuelve una redirección 301 o 308. **Fundamento:** Clase 4, secciones 4.6 y 4.7.

R5 — El contrato funciona de punta a punta

Desde la página publicada, con la consola del navegador abierta, una operación cualquiera completa el intercambio: sale el preflight OPTIONS, sale el POST, vuelve el resultado y la interfaz lo muestra. Sin errores de CORS en la consola.

Además, los dos caminos de error del contrato responden con el código correcto: división por cero → 400 y operación inexistente → 422.

Verificación: interacción en vivo más los comandos de la sección 5.4 contra la API publicada.

Fundamento: Clase 4, secciones 4.8 y 4.9, y el README.md del proyecto.

R6 — Persistencia en base de datos aislada

El historial se guarda en PostgreSQL y sobrevive a un redespliegue del servicio `api`. La base **no tiene ningún puerto publicado**: es alcanzable por su nombre interno desde `api` y por nadie más.

Verificación: tres operaciones, consulta del historial, redespliegue de `api`, nueva consulta —los datos siguen ahí— y un barrido del puerto 5432 desde afuera, que debe darlo cerrado o filtrado.

Fundamento: Clase 5, secciones 5.4 a 5.7.

R7 — Degradación elegante

Quitando `DATABASE_URL` y redesplegando, la calculadora **sigue calculando**. El endpoint de salud informa persistencia: `false` y el historial responde **503**, no 500 ni 404 ni una lista vacía.

Este requisito es el que separa un sistema pensado de uno armado. Una funcionalidad secundaria que no está disponible degrada el servicio; no lo tumba, y lo dice con el código que corresponde.

Verificación: dos peticiones a la API con la variable quitada, y las mismas dos con la variable repuesta. **Fundamento:** Clase 5, secciones 5.6.1 a 5.6.3.

5. Protocolo de verificación

Esta es la batería de comprobaciones que cierra el trabajo, en este orden. El grupo la ejecuta antes de entregar: **si algo de acá falla, todavía no terminaron**.

En los comandos, `tudominio.com` se reemplaza por el dominio del grupo.

5.1. Resolución de nombres

```
dig +short tudominio.com
dig +short api.tudominio.com
```

Ambos devuelven la IP de la VPS. Si alguno devuelve vacío, el registro no está cargado o todavía no propagó.

5.2. Superficie de exposición

```
nmap -Pn -p 22,80,443,3000,5432,8000 tudominio.com
```

Se espera **abierto** en 22, 80 y 443, y **cerrado o filtrado** en todo lo demás. El 5432 es el que decide el R6; el 8000 y el 3000 delatan servicios de desarrollo que quedaron publicados.

5.3. Transporte

```
curl -I http://tudominio.com
curl -I https://tudominio.com
curl -I https://api.tudominio.com/api/salud
```

El primero redirige; los otros dos devuelven 200 sin necesidad de `--insecure`.

5.4. Contrato y errores

```
curl -s -X POST https://api.tudominio.com/api/calcular -H "Content-Type: application/json" -d '{"a":10,"b":4,"operacion":"division"}'
curl -s -o /dev/null -w "%{http_code}\n" -X POST https://api.tudominio.com/api/calcular -H "Content-Type: application/json" -d '{"a":1,"b":0,"operacion":"division"}'
curl -s -o /dev/null -w "%{http_code}\n" -X POST https://api.tudominio.com/api/calcular -H "Content-Type: application/json" -d '{"a":1,"b":2,"operacion":"hackear"}'
```

Se espera el resultado completo en el primero, **400** en el segundo y **422** en el tercero.

5.5. Persistencia y aislamiento

```
curl -s https://api.tudominio.com/api/salud
curl -s https://api.tudominio.com/api/historial
```

El primero informa persistencia: `true`; el segundo devuelve las operaciones, la más reciente primero. Después se redespliega api desde el panel y se repite la segunda petición: los datos siguen.

5.6. Degradación

Se quita DATABASE_URL del servicio api, se redespliega y se repiten las dos peticiones anteriores más un cálculo. El cálculo funciona, la salud informa persistencia: `false` y el historial devuelve **503**. Luego se repone la variable.

EXPERIMENTO

Antes de entregar, hacé esto: pedile a alguien de otro grupo que corra la sección 5.2 contra tu dominio, desde su casa, con su conexión. No desde el aula, y no desde la VPS.

Es la única forma honesta de ver la superficie de exposición: desde adentro del servidor **todo se ve cerrado y todo se ve abierto al mismo tiempo**, según con qué lo mires. El barrido tiene que venir de afuera, y cuanto más afuera, mejor.

6. El informe

Un documento por grupo, en PDF, que documenta el despliegue. No es una crónica de lo que hicieron: es el registro de qué quedó desplegado y de cómo se comprobó que funciona.

Contenido, en este orden:

1. **Carátula.** Materia, comisión, integrantes con legajo, dominio y fecha.
2. **Datos del despliegue.** URL del frontend, URL de la API, proveedor de la VPS y del dominio.
3. **Evidencia de verificación.** Las capturas de la sección 5, con la salida legible.

4. **Incidentes.** Al menos dos problemas reales que hayan tenido, con el síntoma observado, la hipótesis que probaron, la causa encontrada y la corrección. Un informe sin incidentes se lee como un informe incompleto: en cinco clases de infraestructura, algo falló.

⚠ OJO ACÁ

Las capturas tienen que ser **de tu sistema**. Se nota, y no por sagacidad: se nota porque el dominio que aparece en la barra no es el del grupo, porque la IP no coincide con la de la VPS declarada, o porque la marca de tiempo del historial es anterior a la clase en que se creó la base. Una captura ajena no documenta nada: el informe es el registro de tu despliegue, no de otro.

7. Errores frecuentes

Síntoma	Causa habitual	Cómo evitarlo
El dominio resuelve pero el subdominio no	Falta el segundo registro A, o el comodín no cubre el caso	Verificar los dos nombres por separado, no solo el principal
Error de CORS solo en producción	ORIGENES_PERMITIDOS con http en vez de https, o con www de más o de menos	El origen es esquema + host + puerto: copiarlo tal cual de la barra
El historial se vacía en cada despliegue	El servicio de base no tiene volumen	Revisar el almacenamiento del servicio db antes de cargar datos
El puerto 5432 aparece abierto	Se publicó el puerto al crear el servicio	Quitar la publicación; la red interna no la necesita
persistencia: false con la variable cargada	Falta redespargar	Redespargar después de cada cambio de variable
Capturas ilegibles en el informe	Foto de la pantalla con el celular	Captura del sistema operativo, en PNG

8. Entrega

Se entrega por el campus, en un único envío por grupo, con estos elementos:

- ☐ El informe en PDF, nombrado TP-<comisión>-<apellidos>.pdf.
- ☐ La **URL del frontend** publicado.
- ☐ La **URL de la API** publicada.
- ☐ Los nombres y legajos de los integrantes.

El sistema debe permanecer **en línea y sin cambios** desde el momento de la entrega hasta el cierre de la unidad. Un despliegue que se modifica después de entregado se comprueba en el estado en que esté al momento de la verificación.

Hito	Cuándo
Publicación de la consigna	Clase 4
Consultas	Hasta la clase de cierre, por el foro de la unidad
Entrega por el campus	Una semana después de la Clase 5

 DATO

Las consultas van al **foro de la unidad**, no por mensaje privado. No es una formalidad burocrática: el 80 % de las consultas de este práctico son las mismas ocho, y contestarlas en público hace que la novena persona no tenga que esperar. Si la duda incluye una credencial o un dato del servidor, ahí sí va por privado, y de paso ya sabés que ese dato no debería estar en ningún lado público.

9. Preguntas frecuentes

¿Podemos usar el dominio automático que da el panel en vez de uno propio?

No. El módulo entero existe porque hay proveedores de internet en Argentina cuyos servidores DNS no resuelven esos dominios, y porque un nombre propio es lo que permite emitir un certificado que el navegador acepte sin advertencias. El R1 se cumple con dominio propio.

¿La base de datos puede ser la que ofrece el panel como servicio administrado?

Sí, siempre que quede en el mismo proyecto y **sin puerto publicado**. Lo que importa es el aislamiento, no qué botón se apretó para crearla.

Se nos rompió el despliegue el día antes de entregar. ¿Qué hacemos?

Entregar igual, con el informe completo y la sección de incidentes describiendo el estado real. Un grupo que documenta con precisión una falla que no llegó a resolver es mucho más útil que uno que entrega capturas de cuando funcionaba y no avisa.

¿Se puede cambiar de dominio a mitad del práctico?

Sí, pero hay que rehacer los registros DNS, esperar la propagación, reemitir los certificados y actualizar `API_URL` y `ORIGENES_PERMITIDOS`. Son cuatro cosas y las cuatro se olvidan de a una. Si hay que hacerlo, háganlo con margen y vuelvan a correr la sección 5 completa.

¿Cuenta como incidente algo que resolvimos en cinco minutos?

Cuenta si tuvieron que diagnosticarlo. Un error de tipeo no es un incidente; un error de CORS que tardaron cinco minutos en encontrar **porque supieron dónde mirar**, sí, y explicar por qué supieron dónde mirar es exactamente lo que el informe pide.

10. Anexo — Lista de control previa a la entrega

Recórranla con el sistema delante, no de memoria. Cada casilla corresponde a algo que se va a verificar.

Nombres y transporte

- ☐ `dig +short tudominio.com` devuelve la IP de la VPS.
- ☐ `dig +short api.tudominio.com` devuelve la IP de la VPS.
- ☐ `https://tudominio.com` abre sin advertencias del navegador.
- ☐ `https://api.tudominio.com/api/salud` responde estado: ok.
- ☐ `http://tudominio.com` redirige a HTTPS.

Servidor

- ☐ El barrido de puertos desde afuera muestra solo 22, 80 y 443.
- ☐ El acceso SSH por contraseña está deshabilitado.

- ☐ Las claves públicas de los integrantes están cargadas.
- ☐ El firewall está activo con política restrictiva por defecto.

Aplicación

- ☐ Una operación completa funciona desde la página publicada.
- ☐ La consola del navegador no muestra errores de CORS.
- ☐ División por cero devuelve 400; operación inexistente, 422.
- ☐ El historial muestra las operaciones, la más reciente primero.
- ☐ El historial sobrevive a un redesplicue de api.
- ☐ nmap da el puerto 5432 cerrado o filtrado.
- ☐ Sin DATABASE_URL, la calculadora calcula y el historial devuelve 503.

Entrega

- ☐ El informe tiene las capturas de la sección 5, legibles y propias.
- ☐ El informe describe al menos dos incidentes reales.
- ☐ Las URL del frontend y de la API están en el envío del campus.

EXPERIMENTO

El último control, y es el más barato de todos: apagá el servicio api desde el panel y abrí la página. Después prendelo de vuelta.

Lo que veas en esos treinta segundos —qué le muestra la aplicación a alguien cuando la API no está— es lo que va a ver un visitante real el día que algo se caiga. Si lo que aparece es un mensaje entendible, el sistema está terminado. Si es una pantalla en blanco o un “Calculando...” eterno, todavía hay una decisión de diseño sin tomar, y es exactamente la que la sección 5.6 del material pedía tomar.